
Oliver Krauss

Queries and Relationships

RDB | MTD.ba VZ SS26

Hagenberg 16.03.2026

HAGENBERG | LINZ | STEYR | WELS



UNIVERSITY
OF APPLIED SCIENCES
UPPER AUSTRIA

Data Query Language (DQL)

Data Query Language (DQL)

For **querying** the data from the database. Only one command → SELECT.

But lots of functionality:

- Querying with operators
- Sorting
- Grouping
- Joining Tables
- Aggregates
- Functions

SELECT records from the database

Command:

```
1  SELECT column1, column2, ...  
2  FROM table_name  
3  WHERE condition;
```

Example:

```
1  SELECT StudentID, Name  
2  FROM Students  
3  WHERE Department = 'MTD';
```

Example Data - Students

StudentID	Enrolment	Name	Avg_Grade	Department
1	2023-01-15	Luke Skywalker	2.85	SE
2	2024-02-20	Hermione Granger	3.95	IM
3	2023-03-10	Tony Stark	4	MTD
4	2024-04-22	Frodo Baggins	1.75	MBI
5	2023-05-05	Ellen Ripley	3.4	SE
6	2024-06-15	Neo Anderson	2.2	IM
7	2023-07-30	Indiana Jones	1.95	MTD
8	2024-08-18	James Bond	3.1	NULL
9	2023-09-12	Marty McFly	2.45	IM
10	2024-10-25	Leia Organa	3.75	SE

Example Data - Departments

Department	Name
IM	Interactive Media
MBI	Medicine and Bio Informatics
MTD	Media Technology and Design
SE	Software Engineering
HSD	Hardware Software Design
KWM	Communication Knowledge and Media

SELECT

```
1  SELECT StudentID, Name  
2  FROM Students  
3  WHERE Department = 'MTD';
```

Select only a set of columns.

SELECT * returns all columns

WHERE is optional, if omitted all records are returned.

StudentID	Name
3	Tony Stark
7	Indiana Jones

WHERE - Comparison Operators

Operator	Description	Example
=	equals	<code>WHERE Name="Jackie Chan"</code>
>	greater than	<code>WHERE StudentID>101</code>
>=	greater or equal	<code>WHERE StudentID>=101</code>
<	less than	<code>WHERE Enrolment<'2024-01-01'</code>
<=	less than or equal	<code>WHERE Enrolment<='2024-01-01'</code>
!=	not equal (in some DBs <>)	<code>WHERE Department!="MTD"</code>
BETWEEN	value between Range (Number, Date, Text)	<code>WHERE StudentID BETWEEN 100 AND 200</code>
LIKE	search for patterns	<code>WHERE Department LIKE 'MT_';</code>
IN	search for a list of values	<code>WHERE Department IN (MTD, IM, MBI)</code>

WHERE - Logical Operators

Operator	Description	Example
AND	condition a AND b must be true	<code>WHERE Department="MTD" AND Avg_Grade<1.1</code>
OR	condition a OR b must be true	<code>WHERE Department="MTD" OR StudentID<100 OR Avg_Grade<1.1</code>
NOT	condition must not be true	<code>WHERE NOT Name="Jackie Chan"</code>

WHERE - Logical Operator chaining

AND and OR operators can be combined correctly by using brackets:

```
1      SELECT StudentID, Name
2      FROM Students
3      WHERE Avg_Grade<1.1 AND (Department="MTD" OR Department="IM");
```

StudentID	Name
6	Neo Anderson
7	Indiana Jones

WHERE - LIKE and Wildcards

LIKE lets us search for patterns in a specified column using wildcards.

Wildcard	Description	Example
%	zero or more characters	<code>WHERE Name LIKE "Jackie%"</code>
_	any single character	<code>WHERE Department LIKE "M_%"</code>
[]	any character in the brackets	<code>WHERE Department LIKE "M[TB][DI]"</code>
^[^]	any character NOT in the brackets	<code>WHERE Department LIKE "M^[B]^[I]"</code>
[-]	any character in a range	<code>WHERE Department LIKE "M[B-T]%"</code>

Examples for LIKE

Name	Department
Luke Skywalker	SE
Hermione Granger	IM
Tony Stark	MTD
Frodo Baggins	MBI
Ellen Ripley	SE
Neo Anderson	IM
Indiana Jones	MTD
James Bond	
Marty McFly	IM
Leia Organa	SE

```
1  SELECT Name, Department
2  FROM Students WHERE
```

Examples for LIKE

Name	Department
Luke Skywalker	SE
Hermione Granger	IM
Tony Stark	MTD
Frodo Baggins	MBI
Ellen Ripley	SE
Neo Anderson	IM
Indiana Jones	MTD
James Bond	
Marty McFly	IM
Leia Organa	SE

```
1  SELECT Name, Department
2  FROM Students WHERE
```

```
1  Name LIKE "luke%";
```

Name	Department
Luke Skywalker	SE

Examples for LIKE

Name	Department
Luke Skywalker	SE
Hermione Granger	IM
Tony Stark	MTD
Frodo Baggins	MBI
Ellen Ripley	SE
Neo Anderson	IM
Indiana Jones	MTD
James Bond	
Marty McFly	IM
Leia Organa	SE

```
1  SELECT Name, Department
2  FROM Students WHERE
```

```
1  Name LIKE "luke%";
```

```
1  Department LIKE "M_%";
```

Name	Department
Luke Skywalker	SE

Name	Department
Tony Stark	MTD
Frodo Baggins	MBI
Indiana Jones	MTD

DISTINCT

```
1  SELECT DISTINCT Department
2  FROM Students;
```

Department
SE
IM
MTD
MBI

DISTINCT only returns distinct values.
If you use multiple columns, the distinct combination of values is used.

ORDER BY

```
1  SELECT *  
2  FROM Students  
3  ORDER BY Avg_Grade;
```

ORDER BY sorts the results.

Sort can be ascending

(**ORDER BY Avg_Grade ASC**) or descending

(**SORDER BY Avg_Grade DESC**).

Ascending is the default.

StudentID	Enrolment	Name	Avg_Grade	Department
4	2024-04-22	Frodo Baggins	1.75	MBI
7	2023-07-30	Indiana Jones	1.95	MTD
6	2024-06-15	Neo Anderson	2.2	IM
9	2023-09-12	Marty McFly	2.45	IM
...	...	...	...	...

Select specific amount of records

```
1  SELECT *
2  FROM Students
3  ORDER BY Avg_Grade
4  LIMIT 2;
```

LIMIT the amount of returned rows.
Different in other databases (e.g.
(**FETCH FIRST 5 ROWS ONLY** in Oracle)

```
1  SELECT TOP 3 PERCENT *
2  FROM Students
3  ORDER BY Avg_Grade;
```

TOP n items, or percentage if the PERCENT
keyword is given (Microsoft SQL and Access).

StudentID	Enrolment	Name	Avg_Grade	Department
4	2024-04-22	Frodo Baggins	1.75	MBI
7	2023-07-30	Indiana Jones	1.95	MTD

Math Functions

```
1  SELECT Avg_Grade * Avg_Grade
2  FROM Students;
```

Avg_Grade * Avg_Grade
8.122499456405649
15.602500376701357
16
3.0625
11.560000648498544
4.840000209808352
3.802500185966494
9.609999408721933
6.00250023365021
14.0625



Simple math is supported with +, -, *, / and %
ABS, PI, POW, ... also exist.

Other Functions

```
1  SELECT IFNULL(Avg_Grade, 0)
2  FROM Students;
```

IFNULL replaces a null value with a default. Depending on the Database it could also be ISNULL, COALESCE or NVL.

```
1  SELECT CONCAT(UPPER(Name), " ",
                Department)
2  FROM Students;
```

CONCAT adds texts as string, and UPPER returns strings as upper case. Many functions exist and depend on the database used (see [MySQL functions](#))

CONCAT(UPPER(Name), " ", Department)
LUKE SKYWALKER SE
HERMIONE GRANGER IM
TONY STARK MTD
...

WHERE - Sub-Selections

```
1  SELECT StudentID, Name
2  FROM Students
3  WHERE Department NOT IN (SELECT
                             Department from Departments);
```

Values can be replaced with SELECT statements in the WHERE clause

StudentID	Name
8	James Bond

Aggregate Functions

```
1  SELECT MIN(Avg_Grade)
2  FROM Students
3  GROUP BY DEPARTMENT;
```

MIN(Avg_Grade)
3.1
2.2
1.75
1.95
2.85

Aggregate Functions calculate on a set of values and return a single value.

Aggregate Functions

Function	Description
MIN()	Smallest value returned
MAX()	Largest value returned
COUNT()	Counts the number of records
SUM()	Total Sum (only numerical columns)
AVG()	Average of values (only numerical columns)

*all aggregate functions except COUNT ignore NULL values.

Grouping values

```
1  SELECT MIN(Avg_Grade), Name,  
      Department  
2  FROM Students  
3  GROUP BY Department ;
```

MIN(Avg_Grade)	Name	Department
3.1	James Bond	
2.2	Hermione Granger	IM
1.75	Frodo Baggins	MBI
1.95	Tony Stark	MTD
2.85	Luke Skywalker	SE

Groups the selected values into summary rows.

E.g. The best student per department.

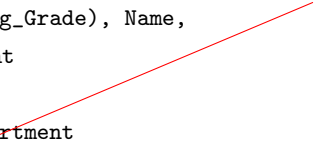
GROUP BY works with multiple columns. E.g.

GROUP BY Department, YEAR(Enrolment)

GROUP BY is mostly used with an aggregate function.

Grouping values HAVING an aggregation

```
1  SELECT MIN(Avg_Grade), Name,  
      Department  
2  FROM Students  
3  GROUP BY department  
4  HAVING MIN(Avg_Grade) < 2.0;
```



MIN(Avg_Grade)	Name	Department
1.75	Frodo Baggins	MBI
1.95	Tony Stark	MTD

HAVING is needed because the WHERE statement cannot filter on aggregates.

HAVING supports all column names in the SELECT even those without aggregate. It is better to use WHERE if you do not use aggregates.

E.g. **HAVING Department="MTD"** should be replaced by **WHERE Department="MTD"**

Aliases

```
1  SELECT s.Department as Dpt
2  FROM Students as s;
```

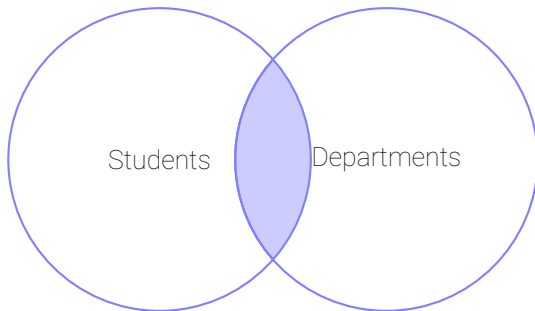
Dpt
SE
IM
MTD
...

Alias for a column (also for aggregates tc.).
Aggregates with space need
AS [My Department] or **AS "My Department"**
depending on the database.

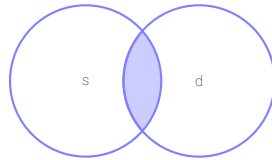
Alias for table.
Helpful for Joins.

Joins

Joins combine two (or more) tables.



Inner Join



```
1  Select s.Name AS Student, d.Name
      AS Department
2  FROM Students AS s
3  INNER JOIN
4  Departments AS d
      ON s.Department
      = d.Department;
```

Join the Students table

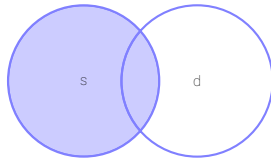
with the Departments table

INNER JOIN records that have **matching values in both tables**

Using the Student.Department to be matched with the Department.Department

Student	Department
Luke Skywalker	Software Engineering
Hermione Granger	Interactive Media
Tony Stark	Media Technology and Design
...	...

Left Join



```
1  Select s.Name AS Student, d.Name
    AS Department
2  FROM Students AS s
3  LEFT JOIN
4  Departments AS d
    ON s.Department
    = d.Department
```

Join the Students table

with the Departments table

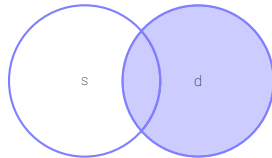
LEFT JOIN records from Students **filling missing values from Departments with null**

Using the Student.Department to be matched with the Department.Department In some

databases it is **LEFT OUTER JOIN**

Student	Department
...	...
Leia Organa	Software Engineering
James Bond	null

Right Join



```
1  Select s.Name AS Student, d.Name
      AS Department
2  FROM Students AS s
3  RIGHT JOIN
4  Departments AS d
      ON s.Department
      = d.Department
```

Join the Students table

with the Departments table

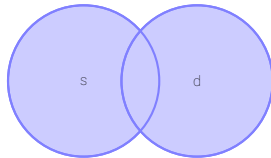
RIGHT JOIN records from Departments **filling missing values from Students with null**

Using the Student.Department to be matched with the Department.Department

In some databases it is **RIGHT OUTER JOIN**

Student	Department
...	...
Leia Organa	Software Engineering
null	Hardware Software Design
null	Communication Knowledge and Media

Full Join



```
1  Select s.Name AS Student, d.Name  
    AS Department  
2  FROM Students AS s  
3  FULL JOIN  
4  Departments AS d  
    ON s.Department  
    = d.Department
```

Join the Students table

with the Departments table

FULL JOIN records from Students with
Departments **filling missing values from
both sides with null**

Using the Student.Department to be matched
with the Department.Department

In some databases it is **FULL OUTER JOIN**, and
some don't support it at all!

UNION

```
1  SELECT column_name(s) FROM table1
2  UNION
3  SELECT column_name(s) FROM table2;
```

Select from A

Combine results with

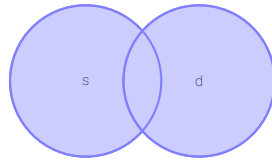
Select from B


```
1  SELECT column_name(s) FROM table1
2  UNION ALL
3  SELECT column_name(s) FROM table2;
```

UNION only selects distinct values.
UNION ALL allows duplicates.

- All SELECT statements must have the same number of columns!
- Columns must have similar datatypes
- Columns must be in the same order in all statements

Full Join as Union



```
1  Select s.Name as Student, d.Name as Department
2  FROM Students as s
3  LEFT JOIN Departments as d ON s.Department = d.
   Department
4
5  UNION
6
7  Select s.Name as Student, d.Name as Department
8  FROM Students as s
9  RIGHT JOIN Departments as d ON s.Department = d.
   Department;
```

Combining the results of a
Left Join
with a
Right Join

Student	Department
...	...
Leia Organa	Software Engineering
James Bond	null
null	Hardware Software Design
null	Communication Knowledge and Media

Cartesian Product

```
1  SELECT Students.Name, Departments
   .Department
2  FROM Students, Departments;
```

Name	Department
Luke Skywalker	IM
Luke Skywalker	MBI
Luke Skywalker	MTD
Luke Skywalker	SE
Luke Skywalker	HSD
Luke Skywalker	KWM
Hermione Granger	IM
...	...

Cartesian Product of Students and Departments

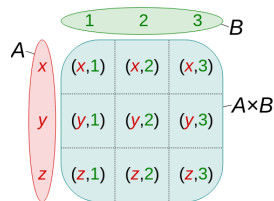


Figure: Source: By Quartl - Own work, CC BY-SA 3.0, <https://commons.wikimedia.org/w/index.php?curid=22436861>